MATH 345 Skeleton Notes

Unit 3: Beyond linear maps: local linearization and first-order optimization

Section 3.6: Directional derivatives, critical points, and gradient descent

---

© 2024–2026 Jacob Denson, Kaitlyn Phillipson, Sebastien Roch. Except where otherwise noted, this work is licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

Directional derivatives

By Linear Approximation, the first-order change in a scalar-valued function near $\mathbf{a}$ is

$$f(\mathbf{a}+\mathbf{h}) - f(\mathbf{a}) \approx \nabla f(\mathbf{a})^T \mathbf{h}.$$

To study change along a unit direction $\mathbf{v}$, restrict the input to the line

$$\mathbf{c}(t) = \mathbf{a} + t\mathbf{v}.$$

The resulting one-variable rate of change is the directional derivative. The chain rule for one-parameter inputs in Chain Rule for One Independent Variable gives the gradient formula below.

Definition

Let f be a scalar-valued function. Then if $\mathbf{v}$ is a unit vector, the [ ] of f in the direction of the vector $\mathbf{v}$, denoted $D_{\mathbf{v}} f$, is defined as

$$D_{\mathbf{v}} f(\mathbf{a}) = \lim_{h \to 0} \frac{f(\mathbf{a}+h\mathbf{v}) - f(\mathbf{a})}{h},$$

provided the limit exists.

The directional derivative represents the rate of change of the function f at the point $\mathbf{a}$ in the direction of the unit vector $\mathbf{v}$. For a function $f(x, y)$ of two variables, the directional derivative $D_{\mathbf{v}} f(\mathbf{a})$ is the slope of the tangent line to the curve obtained by intersecting the surface $z = f(x, y)$ with the plane containing both the line passing through $\mathbf{a}$ with direction vector $\mathbf{v}$, and the z axis.

Remark

Partial derivatives are a special case of directional derivatives, i.e., we have

$$\frac{\partial f}{\partial x_i} = D_{\mathbf{e}_i} f.$$

Despite partial derivatives being a special case of directional derivatives, in most circumstances we can compute directional derivatives using partial derivatives, or equivalently, the gradient.

Theorem

Suppose f is a function differentiable at a point $\mathbf{a}$. Then for each unit vector $\mathbf{v}$, $D_{\mathbf{v}} f(\mathbf{a}) = \nabla f(\mathbf{a}) \cdot \mathbf{v}$.

Why is this true?

Define the path $\mathbf{c}(t) = \mathbf{a} + t\mathbf{v}$ and the single-variable function $g(t) = f(\mathbf{c}(t)) = f(\mathbf{a} + t\mathbf{v})$. Since $\mathbf{c}(0) = \mathbf{a}$, we have $g(0) = f(\mathbf{a})$. Therefore, the definition of the directional derivative gives

$$\begin{aligned} D_{\mathbf{v}} f(\mathbf{a}) &= \lim_{h \to 0} \frac{f(\mathbf{a} + h\mathbf{v}) - f(\mathbf{a})}{h} \\ &= \lim_{h \to 0} \frac{g(h) - g(0)}{h} \\ &= g'(0). \end{aligned}$$

Write $\mathbf{a} = (a_1, \dots, a_n)$ and $\mathbf{v} = (v_1, \dots, v_n)$. The ith coordinate of $\mathbf{c}(t)$ is $c_i(t) = a_i + tv_i$, so $c_i'(0) = v_i$. Because $g = f \circ \mathbf{c}$, the chain rule (Chain Rule for One Independent Variable) gives

$$\begin{aligned} g'(0) &= \sum_{i=1}^{n} \frac{\partial f}{\partial x_i}(\mathbf{c}(0))c_i'(0) \\ &= \sum_{i=1}^{n} \frac{\partial f}{\partial x_i}(\mathbf{a})v_i \\ &= \nabla f(\mathbf{a}) \cdot \mathbf{v}. \end{aligned}$$

Combining the two calculations proves that $D_{\mathbf{v}} f(\mathbf{a}) = \nabla f(\mathbf{a}) \cdot \mathbf{v}$.

The theorem greatly simplifies the computations of directional derivatives.

Example: Directional derivatives and unit directions (U3-LO8)
Let $f(x, y) = x^2 y + y^2$.

Task
Compute the directional derivative $D_{\mathbf{v}} f(1, 2)$, where $\mathbf{v} = (1/\sqrt{2}, 1/\sqrt{2})$, and $f(x, y) = x^2 y + y^2$.

Task

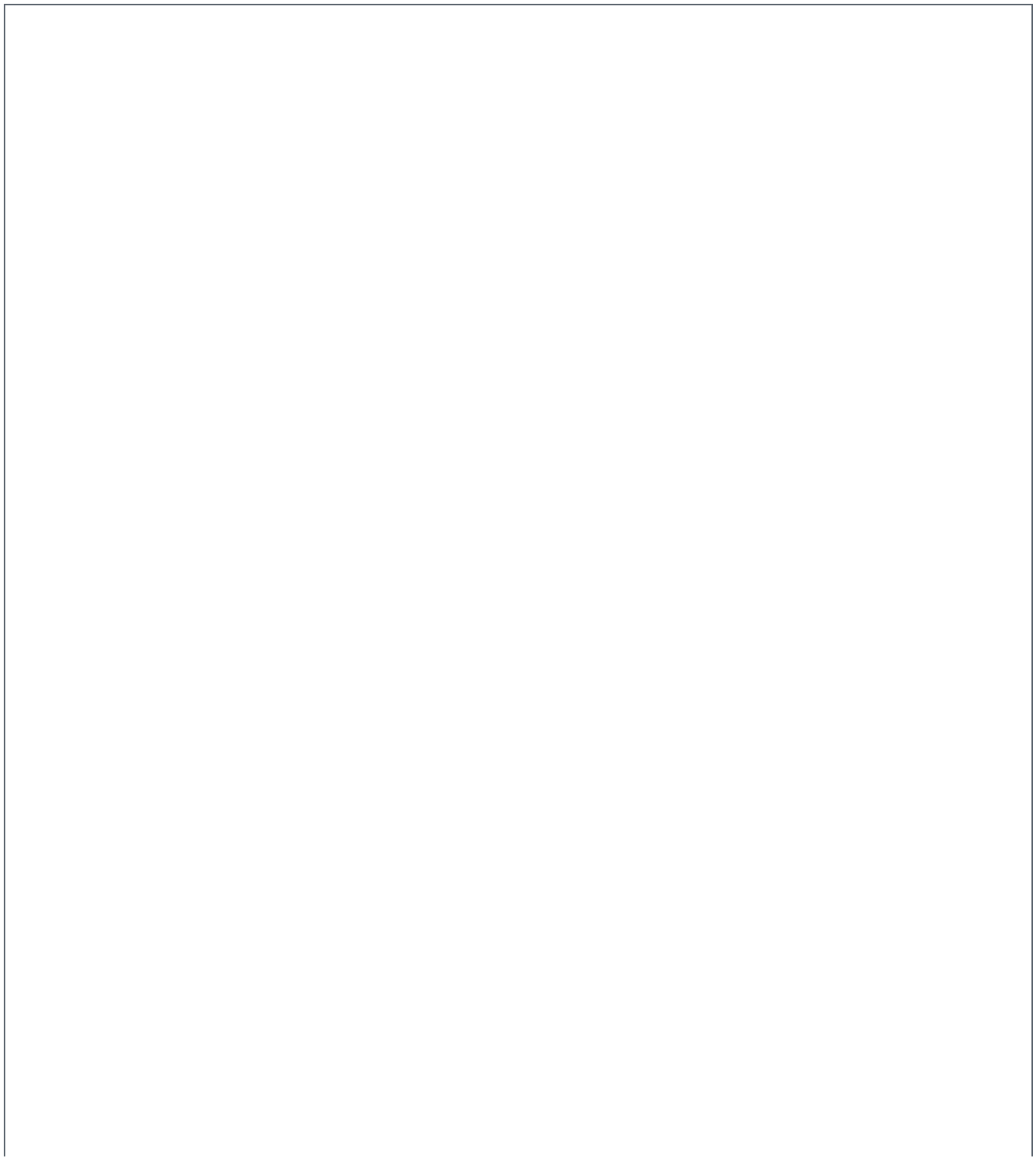
Find the directional derivative of f at the point $(-1, 3)$ in the direction of the vector $(-1, \sqrt{3})$. That is, compute $D_{\mathbf{v}} f(-1, 3)$, where $\mathbf{v}$ is the unit vector in the direction of $(-1, \sqrt{3})$.

Activity: A directional derivative between two points (U3-LO8)

Find the directional derivative of f at $(2, 1)$ in the direction from $(2, 1)$ to $(3, 5)$.

Activity: Directional change toward a specified point (U3-LO8)

Let $f(x, y) = x^2 e^y$. Find the directional derivative of f at $(2, 0)$ in the direction pointing from $(2, 0)$ to $(3, 0)$.



Activity: Alignment with the gradient (U3-LO8, U1-LO1)

Suppose

$$\nabla f(\mathbf{a}) = \begin{bmatrix} 3 \\ 4 \end{bmatrix}.$$

Compute $D_{\mathbf{v}} f(\mathbf{a})$ for each unit vector below:

$$\mathbf{v}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \qquad \mathbf{v}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \qquad \mathbf{v}_3 = \frac{1}{5}\begin{bmatrix} 3 \\ 4 \end{bmatrix}, \qquad \mathbf{v}_4 = -\frac{1}{5}\begin{bmatrix} 3 \\ 4 \end{bmatrix}.$$

Which listed direction gives the greatest increase? Which gives the greatest decrease?

Theorem

Suppose that a function f is differentiable at a point $\mathbf{a}$.

- If $\nabla f(\mathbf{a}) = 0$, then $D_{\mathbf{v}} f(\mathbf{a}) = 0$ for any unit vector $\mathbf{v}$.
- If $\nabla f(\mathbf{a}) \neq \mathbf{0}$, then $D_{\mathbf{v}} f(\mathbf{a})$ is maximized, among unit vectors $\mathbf{v}$, when

$$\mathbf{v} = \frac{\nabla f(\mathbf{a})}{\|\nabla f(\mathbf{a})\|}.$$

It is minimized when

$$\mathbf{v} = -\frac{\nabla f(\mathbf{a})}{\|\nabla f(\mathbf{a})\|}.$$

The maximum value is $\|\nabla f(\mathbf{a})\|$, and the minimum value is $-\|\nabla f(\mathbf{a})\|$.

Why is this true?

If $\nabla f(\mathbf{a}) = \mathbf{0}$, then the theorem gives

$$D_{\mathbf{v}} f(\mathbf{a}) = 0$$

for every unit vector $\mathbf{v}$.

Now suppose $\nabla f(\mathbf{a}) \neq \mathbf{0}$, and let θ be the angle between $\nabla f(\mathbf{a})$ and a unit vector $\mathbf{v}$. By the theorem and Angle between vectors,

$$D_{\mathbf{v}} f(\mathbf{a}) = \nabla f(\mathbf{a}) \cdot \mathbf{v} = \|\nabla f(\mathbf{a})\| \cos\theta.$$

The largest possible value of $\cos\theta$ is 1, attained when $\mathbf{v}$ points in the direction of the gradient. The smallest possible value is -1, attained when $\mathbf{v}$ points in the opposite direction. Therefore the maximum and minimum directional derivatives are

$$\|\nabla f(\mathbf{a})\| \quad \text{and} \quad -\|\nabla f(\mathbf{a})\|,$$

attained at

$$\mathbf{v} = \frac{\nabla f(\mathbf{a})}{\|\nabla f(\mathbf{a})\|} \quad \text{and} \quad \mathbf{v} = -\frac{\nabla f(\mathbf{a})}{\|\nabla f(\mathbf{a})\|},$$

respectively.

Note: Level curves and steepest change

For a function of two variables with continuous partial derivatives and a nonzero gradient at $\mathbf{a}$, the theorem tells us that $\nabla f(\mathbf{a})$ is perpendicular to the level curve through $\mathbf{a}$. Moving along that curve leaves f unchanged, so a tangent direction gives no first-order change in f.

This gives a geometric explanation of the theorem. For motion at unit speed, any tangential component uses part of the motion without contributing to the directional derivative. The greatest increase therefore occurs when the motion is entirely perpendicular to the level curve, in the gradient direction; the greatest decrease occurs in the opposite direction. The dot-product formula identifies which way is uphill: a unit vector in the gradient direction gives the positive rate $\|\nabla f(\mathbf{a})\|$.

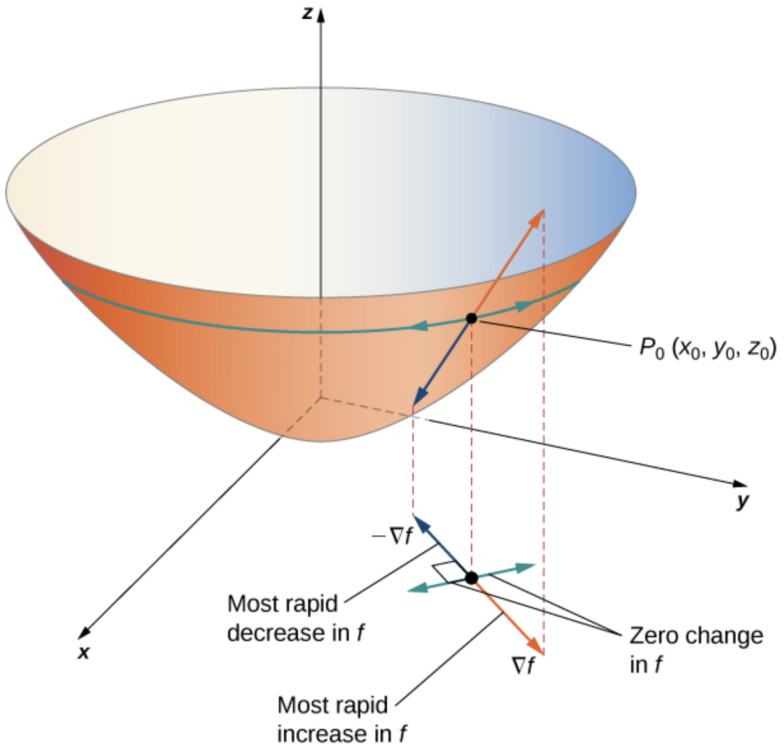


Figure. Gradient directions at a point on a surface.

Activity: Finding the direction of greatest increase (U3-LO8)

Find the direction for which the directional derivative of $f(x, y) = 3x^2 - 4xy + 2y^2$ at $(-2, 3)$ is a maximum. What is the maximum value of the directional derivative?

Activity: A directional derivative in three variables (U3-LO8)

Calculate $D_{\mathbf{u}} f(1, -2, 3)$ in the direction of $\mathbf{v} = -\mathbf{i} + 2\mathbf{j} + 2\mathbf{k}$ for the function

$$f(x, y, z) = 5x^2 - 2xy + y^2 - 4yz + z^2 + 3xz.$$

In Orthogonal projection, the same dot product will be interpreted as a

signed projection coefficient of the gradient onto the chosen direction.

Optimization problems and critical points

Directional derivatives describe first-order change along a chosen direction. We now use them to identify candidates for local extrema and to motivate an iterative method for decreasing a function.
In this unit, we apply multivariate calculus to solve *optimization problems*.

Definition: Optimization problems

Let $f : \mathbb{R}^n \to \mathbb{R}$ be a scalar-valued function, and consider a set $U \subseteq \mathbb{R}^n$. Then we can consider two optimization problems, in which we call f the [ ] of the problem, and U the [ ] of the optimization problem:

- The [ ] of f on U is the problem of finding $\mathbf{x}^* \in U$ such that the value $f(\mathbf{x}^*)$ is *smallest* among all elements of U (so the function is *minimized* on U by $\mathbf{x}^*$).
- The [ ] of f on U is the problem of finding $\mathbf{x}^* \in U$ such that the value $f(\mathbf{x}^*)$ is *largest* among all elements of U (so the function is *maximized* on U by $\mathbf{x}^*$).

Definition: Minima and maxima

Suppose f is a scalar-valued function. A point $\mathbf{a}$ in the domain of f is a

- [ ] if $f(\mathbf{a}) \leq f(\mathbf{x})$ for all $\mathbf{x}$ in the domain of

f. In this case, $f(\mathbf{a})$ is the [ ], or [ ], of the function f.

- [ ] if $f(\mathbf{a}) \geq f(\mathbf{x})$ for all $\mathbf{x}$ in the domain of f. In this case, $f(\mathbf{a})$ is the [ ], or [ ], of the function f.
- [ ] if there is a ball centered at $\mathbf{a}$ such that $f(\mathbf{a}) \leq f(\mathbf{x})$ whenever $\mathbf{x}$ is in that ball and also in the domain of f; in other words, whenever $\mathbf{x}$ is close enough to $\mathbf{a}$ and lies in the domain of f. In this case, we say $f(\mathbf{a})$ is a [ ] for the function f.
- [ ] if there is a ball centered at $\mathbf{a}$ such that $f(\mathbf{a}) \geq f(\mathbf{x})$ whenever $\mathbf{x}$ is in that ball and also in the domain of f; in other words, whenever $\mathbf{x}$ is close enough to $\mathbf{a}$ and lies in the domain of f. In this case, we say $f(\mathbf{a})$ is a [ ] for the function f.

A local minimum or local maximum is also called a [ ].

Remark

The graph of the function $-f$ is obtained by reflecting the graph of the function f in the output axis. This operation turns local and global maxima into local and global minima, and vice versa. So methods for finding

minima for a general function often give methods for finding maxima, which makes the two problems often equivalent.

Definition: Critical points

Suppose f is a scalar-valued function that is differentiable at a point $\mathbf{a}$.

Then we say $\mathbf{a}$ is a [ ] if $\nabla f(\mathbf{a}) = \mathbf{0}$.

For a scalar-valued function of two variables, for example, $f(x, y)$, a critical point is a solution to the two equations $f_x(x, y) = 0$ and $f_y(x, y) = 0$. Just as the derivative of a scalar-valued function of a single variable helps identify local minima and maxima, partial derivatives provide information about possible local minima/maxima.

Theorem

Suppose f is differentiable at an interior point $\mathbf{a}$ of its domain. If $\mathbf{a}$ is a local extremum of f, then

$$\nabla f(\mathbf{a}) = \mathbf{0}.$$

Why is this true?

Let $\mathbf{v}$ be a unit vector, and define

$$g(t) = f(\mathbf{a} + t\mathbf{v}).$$

Because $\mathbf{a}$ is an interior local extremum of f, the function g has a local extremum at $t = 0$. Therefore

$$g'(0) = 0.$$

By the theorem,

$$g'(0) = D_{\mathbf{v}} f(\mathbf{a}) = \nabla f(\mathbf{a}) \cdot \mathbf{v}.$$

This holds for every unit vector $\mathbf{v}$. If $\nabla f(\mathbf{a}) \neq \mathbf{0}$, choose

$$\mathbf{v} = \frac{\nabla f(\mathbf{a})}{\|\nabla f(\mathbf{a})\|}.$$

Then

$$\nabla f(\mathbf{a}) \cdot \mathbf{v} = \|\nabla f(\mathbf{a})\| > 0,$$

a contradiction. Hence $\nabla f(\mathbf{a}) = \mathbf{0}$.

Warning: A necessary condition is not sufficient

For a function differentiable at an interior point $\mathbf{a}$, the theorem says that $\nabla f(\mathbf{a}) = \mathbf{0}$ is a *necessary* condition for a local extremum, but it is *not sufficient*. A critical point need not be a local minimum or a local maximum.

Warning: There need not be exactly one critical point

A differentiable function can have no critical points, one or several critical points, or infinitely many. For example, consider these functions on $\mathbb{R}^2$:

- For $f(x, y) = x$, we have $f_x = 1$ everywhere, so there are *no critical points*.
- For $g(x, y) = (x^2 - 1)^2 + y^2$, the critical-point equations are $4x(x^2 - 1) = 0$ and $2y = 0$. There are *three critical points*: $(-1, 0)$, $(0, 0)$, and $(1, 0)$.
- For $h(x, y) = x^2$, we have $h_x = 2x$ and $h_y = 0$. Thus *every point on the line* $x = 0$ is a critical point: $\nabla h(0, y) = \mathbf{0}$ for every real y.

In the last example, the critical points form an entire line rather than isolated points. On the graph $z = x^2$, they correspond to the line $(0, y, 0)$ highlighted in the figure. Each is a global minimum, since $x^2 \geq 0$ everywhere.

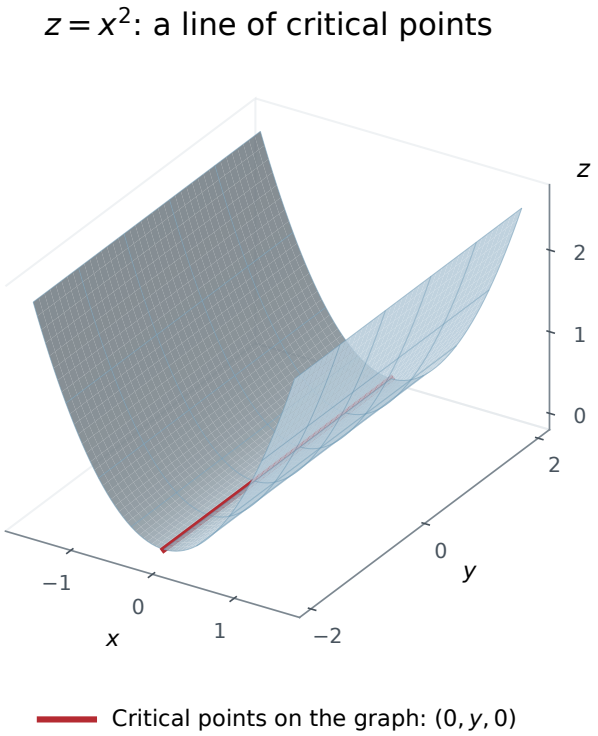


Figure. The graph of $h(x, y) = x^2$. The red line $(0, y, 0)$ corresponds to the infinitely many critical points $(0, y)$ in the input plane. This line continues for all real y.

Example: Critical points of a difference of squares (U3-LO9)

Find the critical points of $f(x, y) = x^2 - y^2$.

Activity: Critical points of a quadratic function (U3-LO9)

Find the critical points of the function

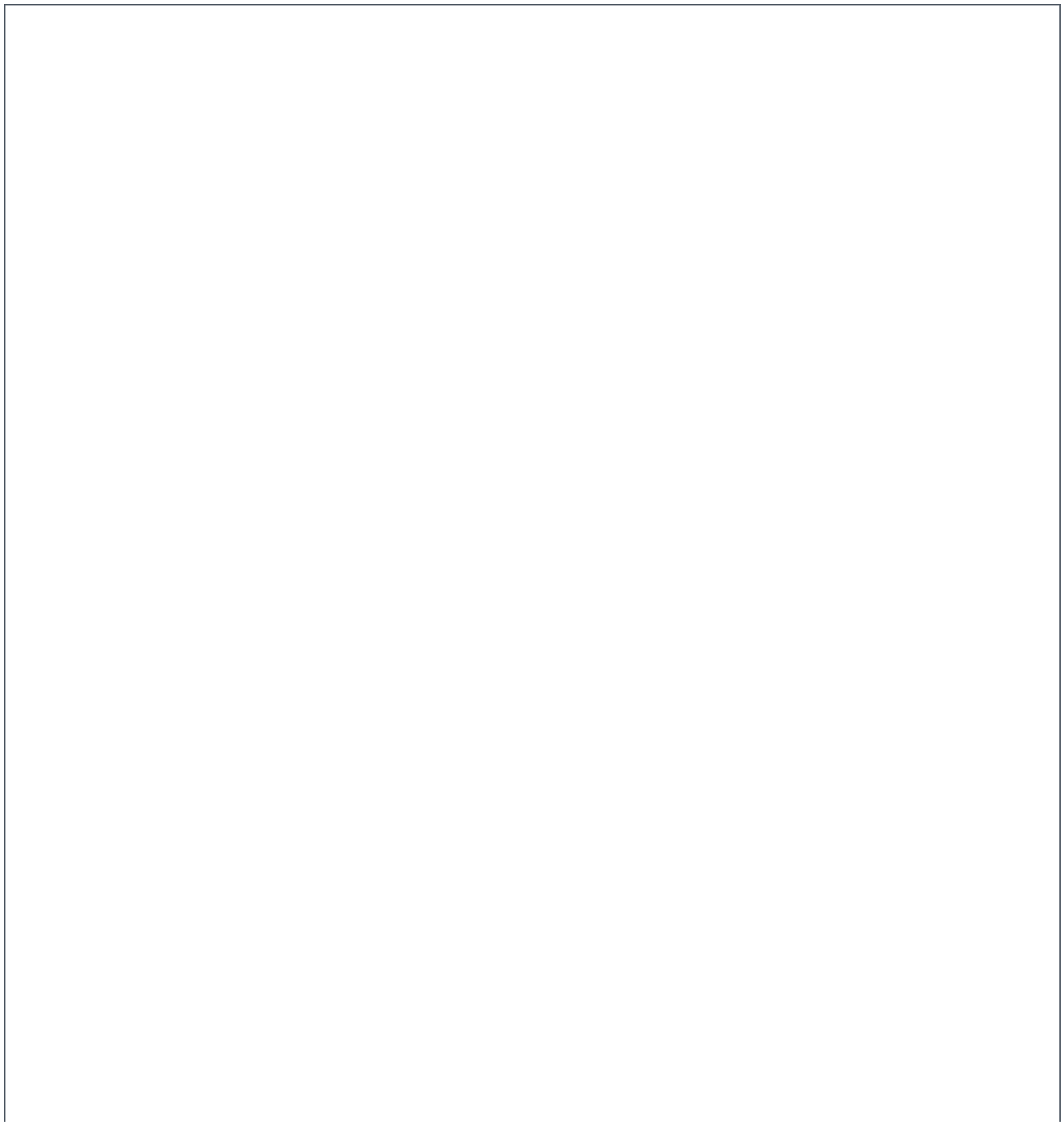
$$f(x, y) = x^2 + y^2 - 2x + 4y.$$

Activity: Critical points via row reduction (U3-LO9, U2-LO2)

Find all critical points of

$$f(x, y, z) = x^2 + xy + yz + z^2 - x - 2y - z.$$

Write the critical-point equations as an augmented matrix, use Gaussian elimination to obtain row echelon form, and finish by back-substitution.



The next critical-point system leads to a quadratic equation. Recall that, when $a \neq 0$, the real solutions of $at^2 + bt + c = 0$ are given by the *quadratic formula*

$$t = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a},$$

provided $b^2 - 4ac \geq 0$. If $b^2 - 4ac < 0$, there are no real solutions.

Activity: Critical points using the quadratic formula (U3-LO9)

Find all critical points of

$$f(x, y) = x^3 + 2xy + y^2 - 2x.$$

Use the quadratic formula to solve the resulting quadratic equation.

When a critical-point equation factors as $(x - a)(y - b) = 0$, at least one factor must be zero: $x = a$ or $y = b$, possibly both. Treat these as two cases, substitute each into the remaining equation, and combine the

solutions, listing any repeated point only once. Dividing by either factor without considering when it is zero can discard valid solutions.

Activity: Critical points by splitting into cases (U3-LO9)

Find all critical points of

$$f(x, y) = (x - 1)^2(y - 2) + y^2 - 8y.$$

Warning: Critical points are candidates

The equation $\nabla f = 0$ finds candidates for local extrema. It does not by itself find boundary extrema, constrained extrema, or absolute extrema. Always combine critical-point information with the domain and boundary conditions.

Activity: Redundant features give a flat direction of the loss (U3-LO6, U2-LO6)

Recall Anatomy of a linear map's redundant-feature example in Two coefficient choices, same predictions (U2-LO1, U2-LO6). Add a third feature to our data by summing the original two feature columns:

Activity: Redundant features give a flat direction of the loss (U3-LO6, U2-LO6) (continued)

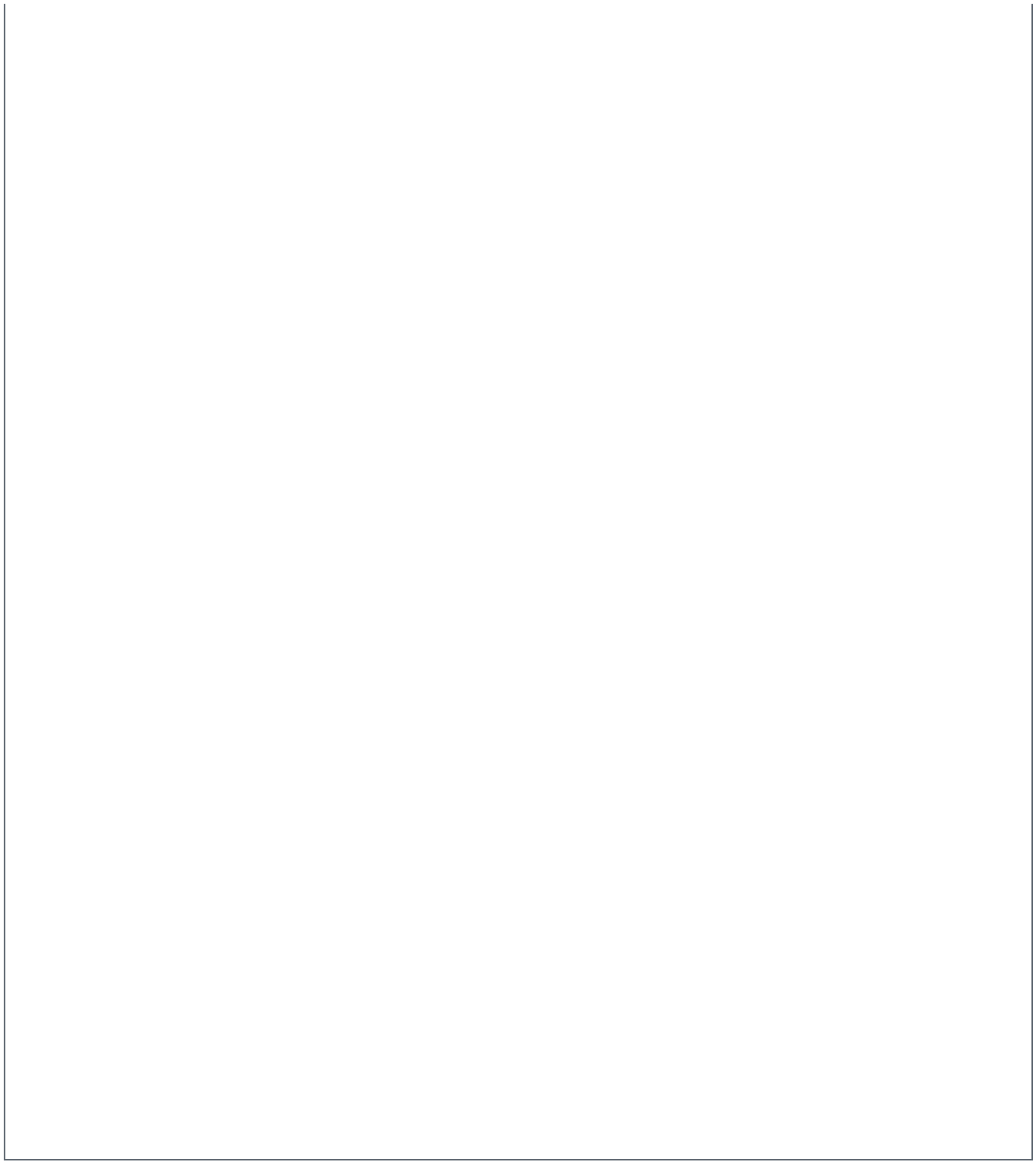
$$\widetilde{X} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 2 \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} 1 \\ 2 \\ 2 \end{bmatrix}.$$

There are now three weights $\theta = (\theta_1, \theta_2, \theta_3)$, with loss

$$\widetilde{\mathscr{L}}(\theta) = \|\widetilde{X}\theta - \mathbf{y}\|^2.$$

The predictions depend only on the two combinations $w_1 = \theta_1 + \theta_3$ and $w_2 = \theta_2 + \theta_3$, so $\widetilde{\mathscr{L}}(\theta) = \mathscr{L}(\theta_1 + \theta_3, \theta_2 + \theta_3)$.

1. Verify that $\mathbf{z} = (-1, -1, 1)$ is in the null space of $\widetilde{X}$.
2. Show that $\widetilde{\mathscr{L}}(\theta + t\mathbf{z}) = \widetilde{\mathscr{L}}(\theta)$ for every real t. Evaluate this constant loss along the line through $\theta_0 = (1, 1, 0)$.
3. What is $\frac{d}{dt}\widetilde{\mathscr{L}}(\theta + t\mathbf{z})$? If a minimizing weight vector is found, can it be the unique minimizing weight vector?



Note: Exact invariance and first-order change

For a linear prediction map $\theta \mapsto \widetilde{X}\theta$, any direction $\mathbf{z}$ satisfying $\widetilde{X}\mathbf{z} = \mathbf{0}$ leaves the predictions, and therefore the loss, unchanged along the entire line $\theta + t\mathbf{z}$. This is the exact invariance illustrated in Redundant features

give a flat direction of the loss (U3-LO6, U2-LO6).
For a differentiable nonlinear map F, the condition $J_F(\mathbf{a})\mathbf{z} = \mathbf{0}$ guarantees only zero first-order change at $\mathbf{a}$. The output $F(\mathbf{a} + t\mathbf{z})$ may still vary with t, as in Interpreting a Jacobian's null space in SymPy (U3-LO5, U2-LO3). Also, equal loss values do not necessarily mean equal predictions: different prediction vectors can have the same squared error, as From predictions to a scalar error function (U3-LO1, U2-LO1) illustrates.

Gradient descent

Example: Critical-point equations in three variables (U3-LO9)

Find the system of equations satisfied by the critical points of the function

$$f(x, y, z) = x^2 e^{yz} + y \sin(xz) + z \cos(xy).$$

Using the calculus we have developed, it is usually straightforward to write the equations that critical points must satisfy. Critical-point equations in three variables (U3-LO9) also shows the problem: those equations may be nonlinear and difficult or impossible to solve exactly. This is different from the linear systems in Unit 2, where Gaussian elimination gave a systematic exact method.

When the critical-point equations cannot be solved directly, one often uses a numerical method. By the theorem, the negative gradient gives the direction of steepest local decrease.

At a current point $\mathbf{x}_k$, choose

$$\mathbf{h} = -\alpha \nabla f(\mathbf{x}_k), \qquad \alpha > 0.$$

By Linear Approximation,

$$\begin{aligned} f(\mathbf{x}_k + \mathbf{h}) - f(\mathbf{x}_k) &\approx \nabla f(\mathbf{x}_k)^T \mathbf{h} \\ &= -\alpha \|\nabla f(\mathbf{x}_k)\|^2. \end{aligned}$$

Unless the gradient is zero, the local model predicts a decrease. This is a local prediction, not a convergence theorem. It motivates the gradient-descent update

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \nabla f(\mathbf{x}_k).$$

The gradient-descent algorithm repeats this local step.

1. Start at an initial point $\mathbf{x}_0$.
2. Compute the gradient $\nabla f(\mathbf{x}_0)$.
3. Move in the direction of the negative gradient: set $\mathbf{x}_1 = \mathbf{x}_0 - \alpha_0 \nabla f(\mathbf{x}_0)$, where $\alpha_0 > 0$ is a step size (or learning rate) parameter.
4. Repeat steps 2-3 with step sizes $\alpha_1, \alpha_2, \ldots$ to successively define points $\mathbf{x}_2 = \mathbf{x}_1 - \alpha_1 \nabla f(\mathbf{x}_1)$, $\mathbf{x}_3 = \mathbf{x}_2 - \alpha_2 \nabla f(\mathbf{x}_2)$, and so on, with the hope that these points converge to a local minimum.

In data and modeling problems, an objective function being minimized is often called a loss.
Intuitively, gradient descent works because each successive value $\mathbf{x}_k$ is obtained by shifting the previous value $\mathbf{x}_{k-1}$ in the direction in which the function f decreases the fastest. Gradient descent may converge to a local minimum under additional assumptions on the function and the learning rates. The examples below illustrate possible behavior; they do not prove convergence.

Activity: Three gradient-descent steps (U3-LO9)

Find an approximate minimum of the function $f : \mathbb{R}^2 \to \mathbb{R}$ defined by

$$f(x, y) = x^2 - 3xy + 3y^2 + 5y + 2x$$

using gradient descent. Use the starting point $\mathbf{x}_0 = (0, 0)$, a constant step size of 0.1, and stop after computing $\mathbf{x}_3$. You may use a calculator to perform these computations.

First gradient descent steps

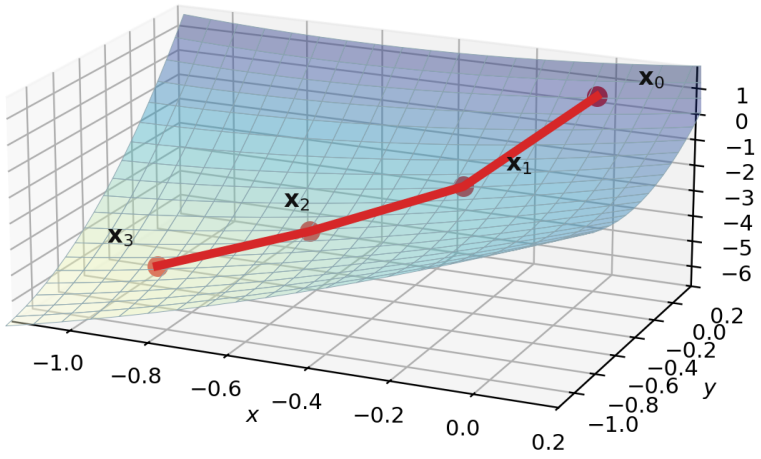


Figure. Surface plot showing the first gradient descent steps from the origin.

Activity: From a symbolic gradient to a descent step (U3-LO9)

Use SymPy to check the first step of the preceding calculation for

$$f(x, y) = x^2 - 3xy + 3y^2 + 5y + 2x.$$

We compute the gradient as the transpose of the Jacobian, as in Jacobian, gradient, and Hessian, evaluate it at the starting point, and apply the gradient-descent update. The learning rate `sp.Rational(1, 10)` represents 0.1 exactly.

Activity: From a symbolic gradient to a descent step (U3-LO9) (continued)

Since `.jacobian()` is a `Matrix` method, we wrap `f` in the one-entry matrix `sp.Matrix([f])`. The call `.jacobian([x, y])` produces the 1×2 row of partial derivatives; `.T` transposes it into the 2×1 gradient column.

```
import sympy as sp

x, y = sp.symbols("x y")
f = x**2 - 3*x*y + 3*y**2 + 5*y + 2*x
grad_f = sp.Matrix([f]).jacobian([x, y]).T

p0 = sp.Matrix([0, 0])
g0 = grad_f.subs({x: p0[0], y: p0[1]})
alpha = sp.Rational(1, 10)
p1 = p0 - alpha*g0
f0 = f.subs({x: p0[0], y: p0[1]})
f1 = f.subs({x: p1[0], y: p1[1]})

print(grad_f)
print(g0)
print(p1)
print(f0, f1)
print((g0.T * (p1 - p0))[0], f1 - f0)
```

Output:

```
Matrix([[2*x - 3*y + 2], [-3*x + 6*y + 5]])
Matrix([[2], [5]])
Matrix([[-1/5], [-1/2]])
0 -241/100
-29/10 -241/100
```

In the last line, `g0.T * (p1 - p0)` is a 1×1 matrix. The index `[0]` extracts its scalar entry, the local linear prediction for the change in f.

Activity: From a symbolic gradient to a descent step (U3-LO9) (continued)

1. What is the difference between `grad_f` and `g0`? What shapes must `p0`, `g0`, and `p1` have for the update to make sense?
2. Write the numerical vector update that produces `p1`. Did the function value decrease? Compare the predicted change with the actual change, using the scalar linear approximation from Linear Approximation.
3. Predict `grad_f.subs({x: p1[0], y: p1[1]})`. Is `p1` a critical point? Explain which symbolic expression can be reused at the next step and which evaluated vector must be recomputed.
4. If `alpha` is changed to `1` and the subsequent lines are rerun, what are `p1` and `f1`? What does this comparison show about the learning rate?

Activity: A gradient-descent step in NumPy (U3-LO9)

Repeat the computation in From a symbolic gradient to a descent step (U3-LO9) using NumPy for

$$f(x, y) = x^2 - 3xy + 3y^2 + 5y + 2x.$$

Here `f` and `grad_f` are Python functions evaluated at numerical inputs. We supply the partial-derivative formulas found in the preceding activity; NumPy does not differentiate the expression in this code. Points and gradients are represented by one-dimensional arrays.

```
import numpy as np

def f(p):
    x, y = p
    return x**2 - 3*x*y + 3*y**2 + 5*y + 2*x

def grad_f(p):
    x, y = p
```

Activity: A gradient-descent step in NumPy (U3-LO9) (continued)

```
    return np.array([2*x - 3*y + 2, -3*x + 6*y + 5])

p0 = np.array([0.0, 0.0])
g0 = grad_f(p0)
alpha = 0.1
p1 = p0 - alpha*g0
f0 = f(p0)
f1 = f(p1)

print(p0.shape, g0.shape, p1.shape)
print(g0)
print(p1)
print(np.round(f0, 2), np.round(f1, 2))
print(np.round(g0 @ (p1 - p0), 2), np.round(f1 - f0, 2))
```

Output:

```
(2,) (2,) (2,)
[2. 5.]
[-0.2 -0.5]
0.0 -2.41
-2.9 -2.41
```

For one-dimensional arrays, `@` computes a dot product and returns a scalar. The calls to `np.round` round the displayed scalar results to two decimal places; they do not change the stored point or gradient.

1. Compare `grad_f` in the two activities. Which version computes derivatives symbolically, and which evaluates derivative formulas supplied by us? How does `grad_f(p0)` replace the SymPy substitution step?
2. Why do the NumPy arrays have shape `(2,)`, while the SymPy point and gradient matrices have shape `(2, 1)`? Explain why

Activity: A gradient-descent step in NumPy (U3-LO9) (continued)

`g0 @ (p1 - p0)` replaces `(g0.T * (p1 - p0))[0]`. Would `g0 * (p1 - p0)` give the same result in NumPy?

3. Write the vector update that produces `p1`, and compare the new point, function value, and predicted change with the exact SymPy results. What changes when `sp.Rational(1, 10)` is replaced by `0.1`?

4. Predict `grad_f(p1)` and compare it with the corresponding SymPy substitution. Is `p1` a critical point? In each version, what can be reused and what must be evaluated again for the next step?

5. Starting again at `p0`, change `alpha` to `1.0`. Find the new point and function value in both versions. Is the increase in the function value a floating-point error, or a consequence of the learning rate?

Remark

The iteration is cumbersome by hand but easy to repeat on a computer when the gradient is available.

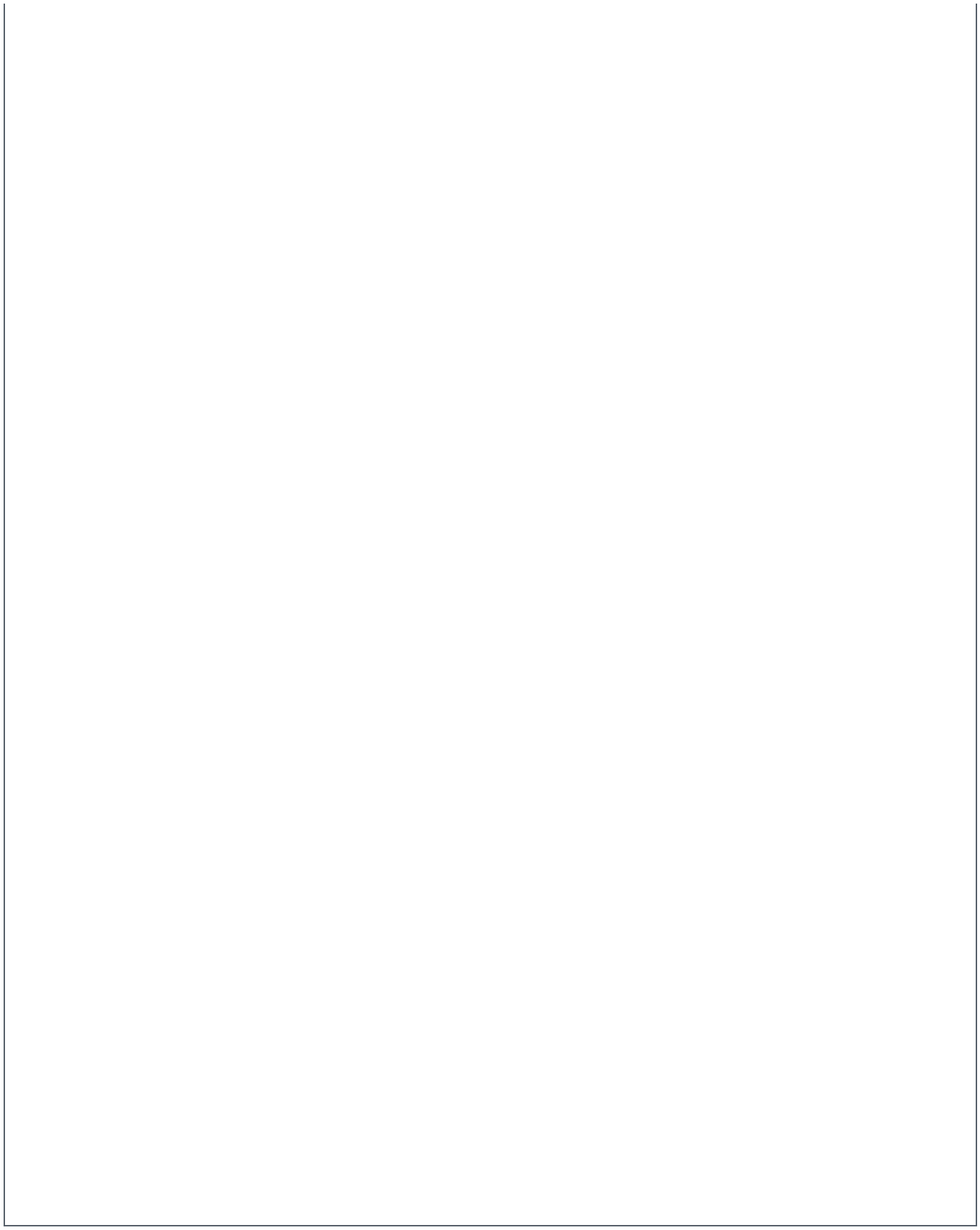
Activity: Improving a model's predictions with one step (U3-LO8, U3-LO9)

Return to the three observations from From predictions to a scalar error function (U3-LO1, U2-LO1) and the loss

$$\mathscr{L}(w_1, w_2) = (w_1 - 1)^2 + (w_2 - 2)^2 + (w_1 + w_2 - 2)^2.$$

At the starting weights $\mathbf{w}_0 = (1, 1)$, the loss is 1 and the gradient is $(0, -2)$. In this model, a training step adjusts the weights while the features X and observations $\mathbf{y}$ remain fixed.

1. Find the unit direction of greatest decrease at $\mathbf{w}_0$ and its directional derivative. Take one gradient-descent step with learning rate $\alpha = 1/10$.
2. Compute the new prediction vector, residual, and loss. Compare the actual change with the first-order prediction $-\alpha\|\nabla\mathscr{L}(\mathbf{w}_0)\|^2$.
3. Evaluate the gradient at the new weights. Has the step reached a critical point?
4. Starting again at $\mathbf{w}_0$, what happens if $\alpha = 1$? Explain why the same descent direction can give an increase in loss.



These examples use derivatives to improve a fit and interpret directions that leave it unchanged. Unit 4 develops the general least-squares

method for finding a best-fitting prediction vector and its geometric interpretation.